

Rapport n°2



- Membres :
 - Yassine ELKHALKI
 - Léna RUHAULT
 - Ismail CETINOVA
 - Dépôt Github :
<https://github.com/YassouSensai/MonCNAM.git>
-

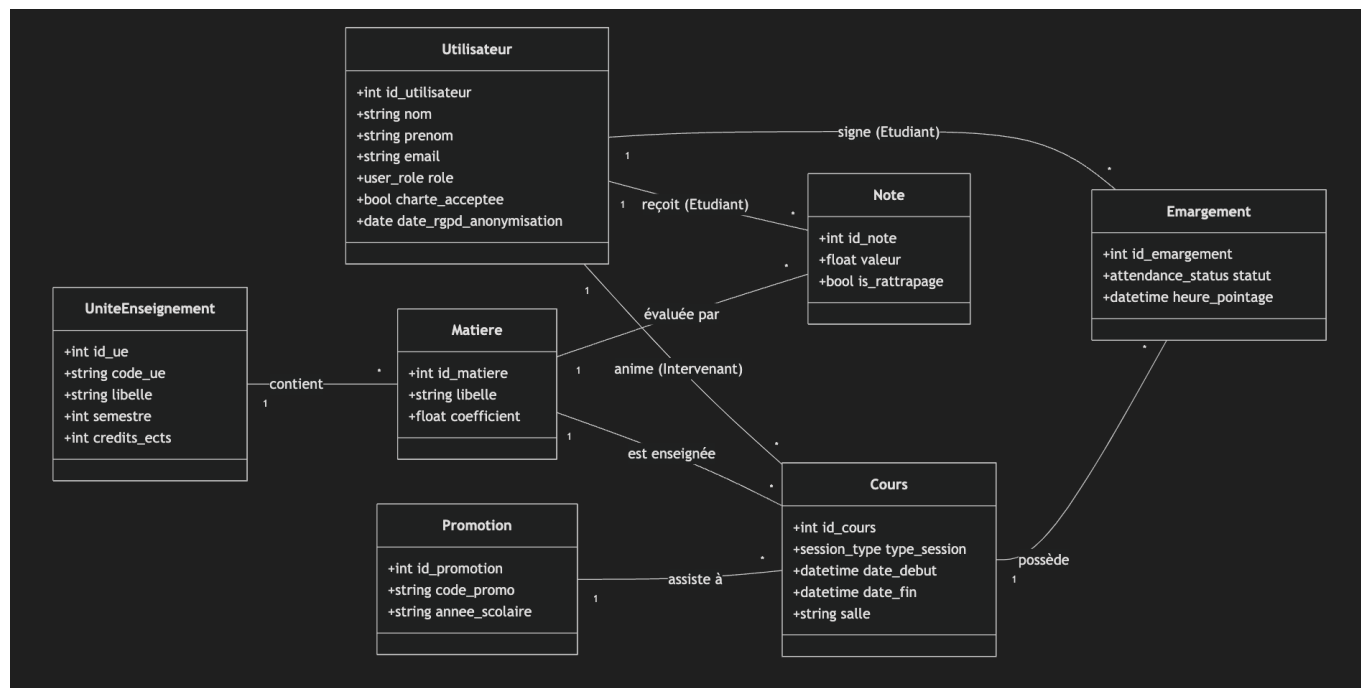
Sommaire

1. Spécifications fonctionnelles et techniques
 - Diagramme de classe / Entité-relation
 - Diagramme d'état
 - Diagramme de séquence
 - Diagramme d'architecture
2. Choix technologiques
 - Frontend
 - Backend
 - Base de données
 - Infrastructure

Spécifications fonctionnelle et technique

Diagramme de classe / Entité-relation

Le diagramme suivant illustre l'architecture logique de la plateforme MonCNAM. Il met en évidence la séparation entre le référentiel statique, c'est à dire les UEs ou encore les matières et les données dynamiques comme les cours, les notes et les émargements.



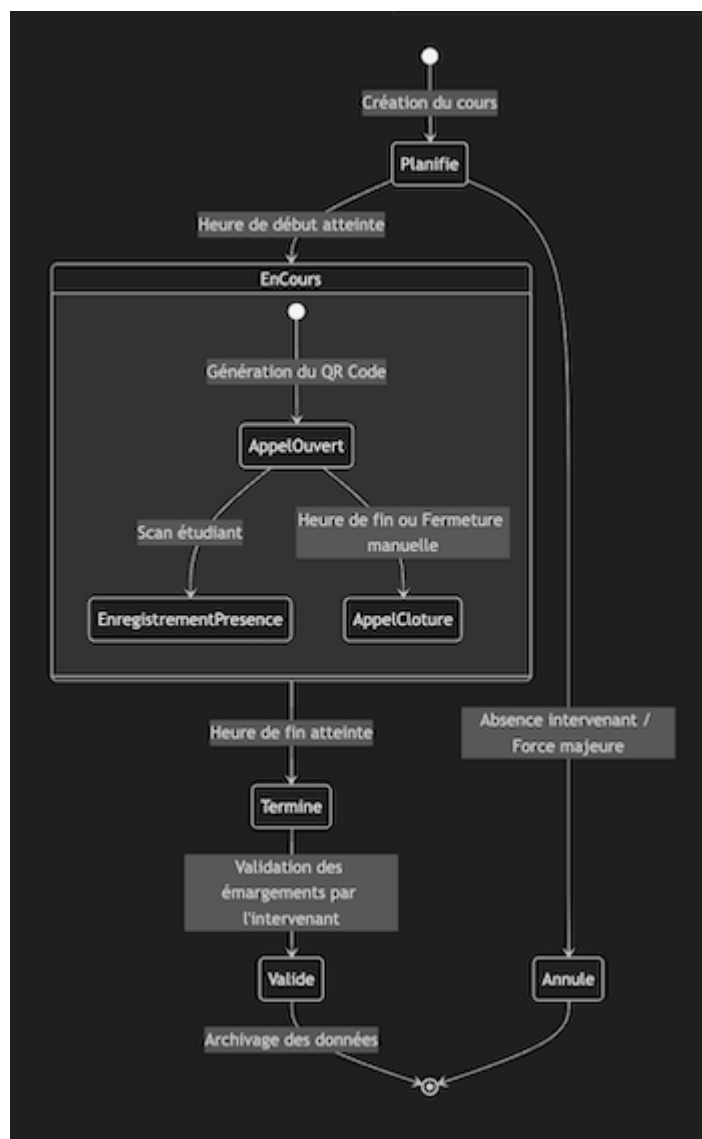
Spécification des principales entités

- **Utilisateurs** : Il s'agit de l'entité centrale qui va gérer l'authentifications, les droits ainsi que l'accès aux fonctionnalités en fonction du rôle de l'utilisateur.
- **Promotion** : Regroupe les élèves par année scolaire et par code (ex : DICASI-2026). C'est la promotion qui est concerné par un emplois du temps.
- **UniteEnseignement & Matiere** : Les deux entités pour les gestion des enseignements. D'un côté le regroupement thématique des matières par semestre et crédits ECTS. Et de l'autre, tout(e)s les matières/enseignements.
- **Cours & Emargement** : Pour gérer le déroulement des cours et les présences.

Nous avons également prévu un champ ***date_rgpd_anonymisation*** pour gérer le cycle de vie des données personnelles (notamment en fin de formation par exemple)

Diagramme d'état

Le diagramme d'état ci-dessous modélise le cycle de vie complet d'une séance de cours, de sa planification initiale à sa validation. Cette modélisation définit les fenêtres temporelles durant lesquelles les interactions seront réalisés tout en garantissant l'intégrité des données d'émargement.

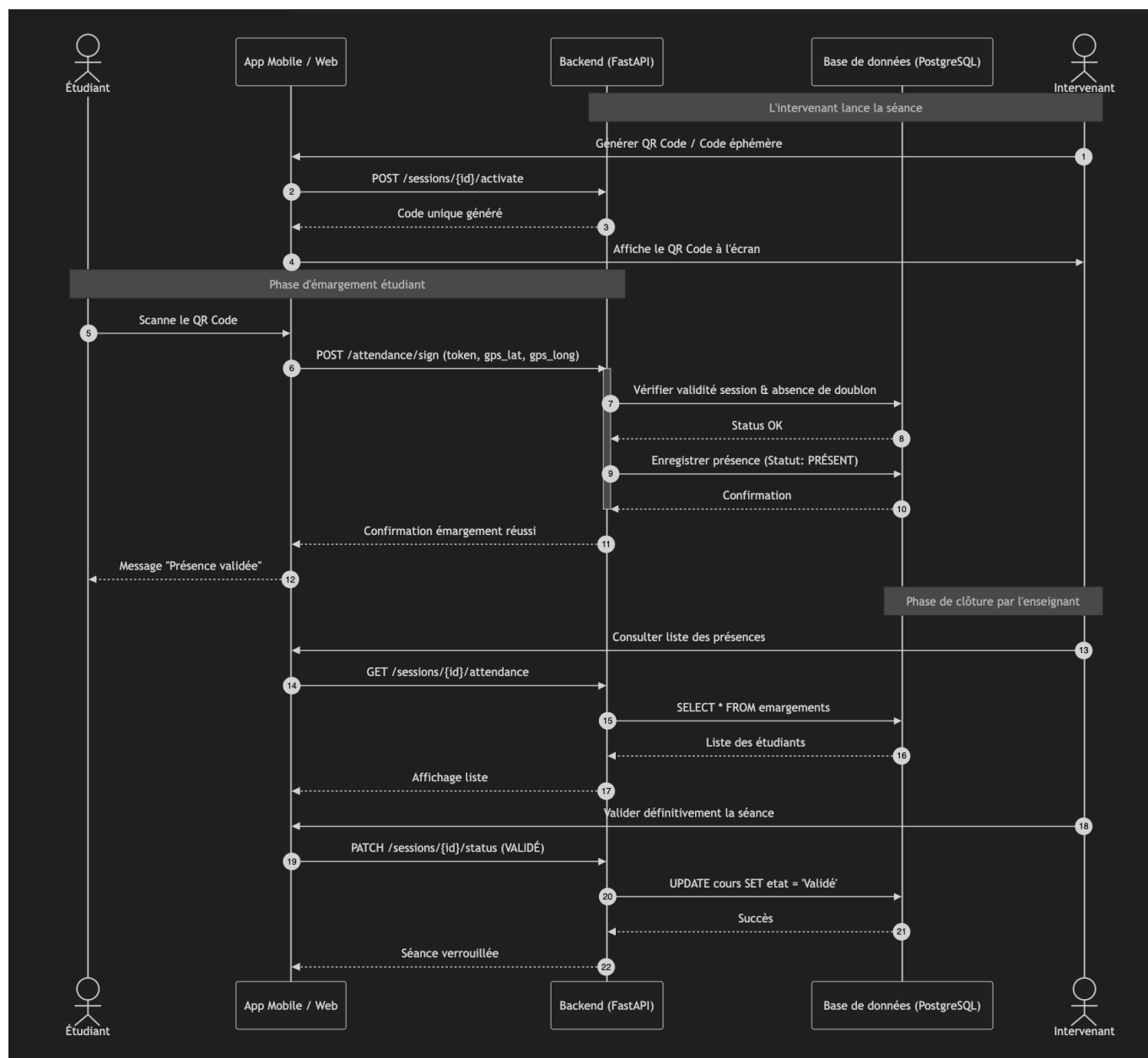


Description des états

- **Planifié** : Cet état correspond à l'existence du cours dans l'emploi du temps avant l'heure de début.
- **En cours** : Il s'agit de la phase active. C'est un état composite qui gère la logique d'émargement via scan d'un qr code généré par l'enseignant. L'enseignant pourra compléter l'emargement à la fin du cours si certains élèves n'ont pas scanné le qr code (état terminé), puis valider l'émargement (état validé).
- **Annulé** : En cas de force majeure ou enseignant absent, alors le cours est annulé.

Diagramme de séquence

Ce diagramme illustre les interactions dynamiques entre les acteurs et la plateforme MonCNAM lors du processus d'émargement numérique.

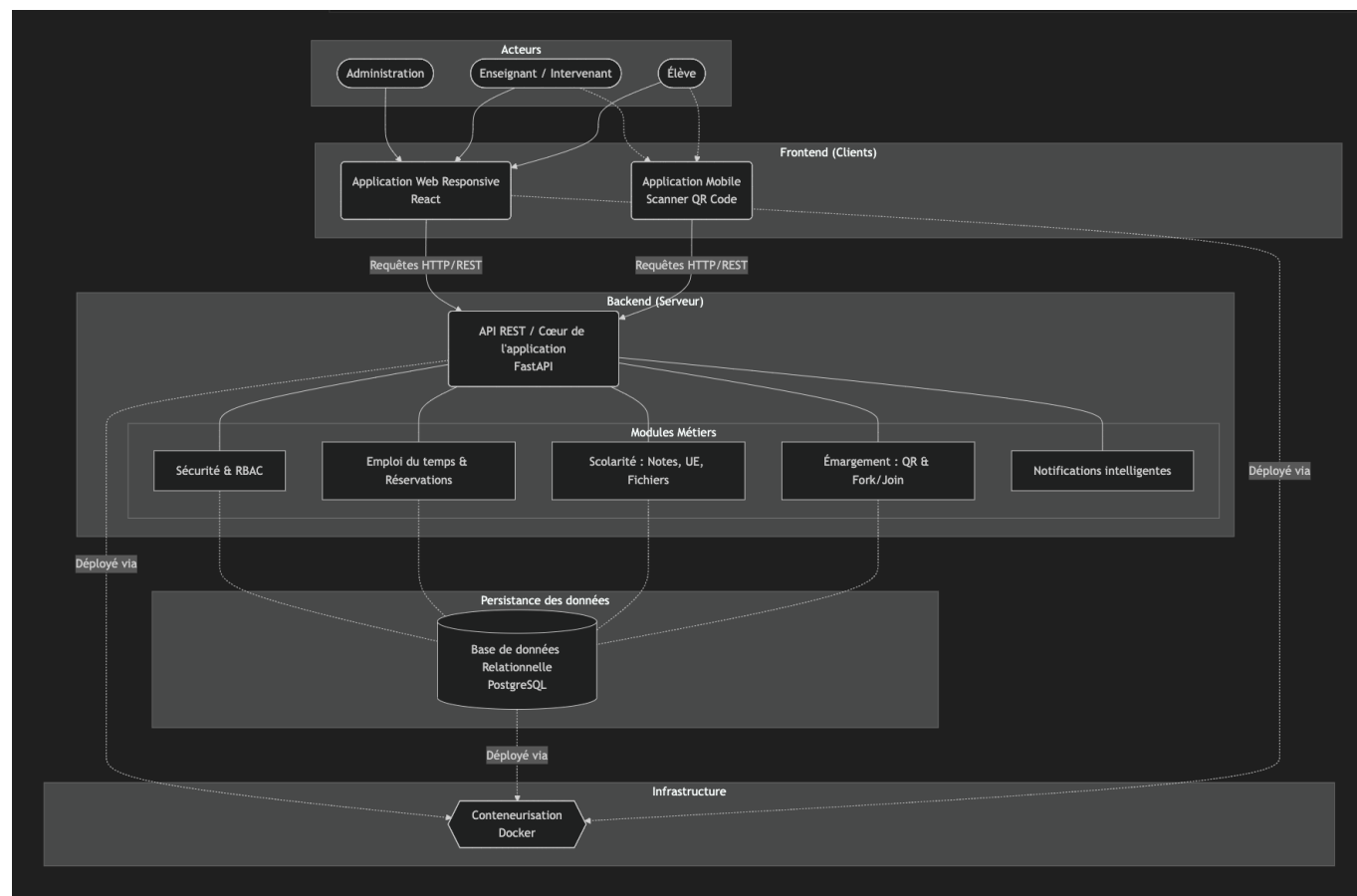


Déroulement de la séquence

- **L'initialisation** : L'intervenant génère via le Backend un jeton unique (QR Code) qui la session courante.
- **L'émargement** : L'étudiant scanne le code. L'API vérifie immédiatement la validité du jeton, s'assure que l'étudiant n'a pas déjà émargé (unicité) et valide la présence en base de données. L'intégration de coordonnées GPS optionnelles permet de contrer la fraude (scan à distance), mais on ne sais pas encore si ce sera implémenté.
- **La validation** : L'intervenant récupère la liste mise à jour en temps réel. La clôture de la séance verrouille les données pour empêcher toute modification ultérieure.

Diagramme d'architecture

MonCNAM repose et reposera sur une architecture moderne type client-serveur



L'architecture est scindée en plusieurs couches indépendantes communiquant via des requêtes HTTP/REST, ce qui permet d'isoler la logique métier (Backend) de l'interface utilisateur (Frontend). L'ensemble est encapsulé dans une infrastructure conteneurisée pour garantir l'uniformité entre les environnements de développement et de production.

Choix technologiques

Afin de répondre aux exigences de fonctionnement mais également de modernisation de MonCNAM (Galao actuellement), nous avons fait des choix avec des technologies que l'on connaît déjà et d'autres non.

Interface Utilisateur (Frontend) : React

Nous avons opté pour la bibliothèque React. Son architecture orientée composants permet de développer une interface riche et réutilisable (ex: les grilles d'emploi du temps). De plus, React facilite la création d'une application "Responsive" capable de s'adapter aussi bien aux écrans d'ordinateur de l'administration qu'aux smartphones des étudiants (nécessaire pour la fonctionnalité de scan). Nous avons la possibilité de découvrir cette technologie en cours de SI WEB.

Logique Métier (Backend) : FastAPI (Python)

Le choix de FastAPI s'impose pour notre backend. Ce framework Python est particulièrement performant grâce à sa gestion native de l'asynchronisme. Ce point est critique pour notre système d'émargement : en début de cours, l'API va subir des pics de charge importants (plusieurs dizaines d'étudiants scannant un QR code dans la même minute). FastAPI permet de traiter ces flux simultanés sans bloquer le serveur. De plus, il génère automatiquement la documentation de l'API (Swagger), facilitant la coordination entre le développement Front et Back. Nous sommes tous les trois à l'aise avec Python

Base de données : PostgreSQL

Pour la persistance des données, PostgreSQL a été retenu. Il s'agit d'un Système de Gestion de Base de Données reconnu pour sa fiabilité et de plus en plus utilisé en entreprise (par nous aussi). La nature des données traitées (notes d'étudiants, présences ayant un impact sur la diplomation, contraintes d'unicité) exige une structure relationnelle forte et intégrée (difficile d'utiliser une BD NoSQL)

Infrastructure : Docker

L'ensemble des services (Frontend, Backend, Base de données) sera conteneurisé via Docker. Ce choix répond à une contrainte forte de maintenabilité : il garantit que la solution fonctionnera à l'identique sur nos postes de développement locaux, lors des tests, et pour la potentielle mise en disposition sur un serveur privé nous appartenant ou bien sur un cloud public à moindre coût.